ELTE IK - Computer Science BSc

Final Exam Topics

6. Artificial intelligence

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Artificial intelligence

The relationship between AI problems and the pathfinding task. Modeling techniques (e.g. state space model, decomposition model). Heuristic pathfinding algorithms: local searches (hill climbing, tabu search, simulated annealing), backtracking search, heuristic graph search procedures (A, A*, A^C, B algorithms). Two-player games.

1 Introduction

AI researches, develops, and systematizes the principles, methods, and techniques useful for the computer reproduction of intelligent thinking. Its tasks are hard, because their problem space is huge, and finding the solution, in the absence of sufficient intuition, can lead to a combinatorial explosion. The software behaves intelligently and uses specific tools. The representation is well-considered for modeling the task, and the algorithms are efficient, reinforced with heuristics.

2 Pathfinding problems

Many AI tasks can be formulated as pathfinding problems by specifying, based on the model of the task, an edge-weighted directed graph in which the paths leading from a given vertex to a given vertex represent the individual solutions of the task. This is also usually called the *graph representation* of the task, which includes a so-called *δ-graph* (an edge-weighted directed graph where the number of edges leaving a vertex is finite, and a positive lower bound δ can be given for the cost of the edges), the start vertex designated in it, and one or more goal vertices. In this representation graph we look for a path starting from a start vertex and running to a goal vertex, in some cases a cheapest such path.

3 Modeling techniques

3.1 State space representation

State space representation is one possible (but not the only) method for modeling tasks, which can then naturally also be formulated as a graph representation. It has four elements:

- *State space*, which is the set of possible values (states) of the data (object) at the forefront of the problem. Often a base set restricted by a suitable invariant.
- *Operations* (precondition + effect), which lead from state to state.
- *Initial state*(s) or an initial condition describing them.

- *Goal state*(s) or goal condition.

The *state graph* (a special representation graph) contains the states as vertices and the effects of the operations as edges. The state space is not identical to the problem space, since the elements of the problem space are the paths (operation sequences) leading from the start vertex, not the states (vertices). The solution is an element of the problem space, which is an operation sequence that leads from a start vertex to a goal vertex.

3.2 Decomposition model

The essence of problem decomposition is that we break a task into subtasks, then detail those further until we obtain obviously solvable tasks. For the representation we have to specify:

- a general description of the subproblems of the task,
- the original problem,
- the simple problems, about which it can easily be decided whether they are solvable or not, and
- the decomposition operations:

Decomposition operations are very hard to find:

- It is not certain that we will find them.
- False decomposition operations.
- Not every task is decomposable.

Recognizing a simple problem is also not unambiguous. Reading out the solution is also not obvious. In the (R, s, T) graph representation belonging to a decomposition representation

- $R = (N, A, c)$ is an AND/OR graph (a decomposition graph), where
- N symbolizes the subproblems,
- A the decomposition operations,
- c their costs,
- s the original problem,
- T denotes the simple problems.

The solution of the problem means finding an $s \to M \subseteq T$ hyperpath not containing an ordinary directed cycle, the so-called solution graph. The solution of the original problem can be extracted from this solution graph. The cost of the solution usually does not depend on the cost of the solution graph, so producing the optimal solution graph is not a goal.

4 Searches

The parts of a general *search system*: the *global workspace* (the memory of the search), the *search rules* (which change the content of the memory), and the *control strategy* (which at a given moment chooses a suitable rule). The control strategy has a general, primary element (this can be non-modifiable or modifiable), it can have a secondary element (one exploiting the peculiarities of the applied representation model), and an element building on the concrete task. The latter is the *heuristic*, the extra knowledge derived from the concrete task, which we build directly into the control strategy for the purpose of improving effectiveness and efficiency.

5 Local searches

Local searches store only a single current vertex and its narrow neighborhood in the global workspace. Their search rules replace the current vertex at each step with a preferably "better" child vertex chosen from among its neighbors. To decide "betterness", their control strategy uses a *fitness function*, which gives a better value for a vertex the closer it is to the goal. Since the search "forgets" where it came from, the decisions cannot be undone; this is a *non-modifiable control strategy*. Tasks solvable with local search are those where a locally made bad decision does not exclude finding the goal. For this either a strongly connected representation graph or a goal function built on a good heuristic is needed. Typical applications: searching for an element with a given property, searching for the optimum of a function.

- *Hill climbing algorithm*: At each step it moves to the best child of the current vertex, but excludes moving back to the parent. It gets stuck in a dead end (no edge leads out of the current vertex), and can fall into an infinite loop along cycles if the fitness function is not perfect.
- *Tabu search*: Besides the current vertex (n), it also keeps track of the vertex that has proven best so far (n*) and the last few visited vertices; this is the (queue-property) tabu set. At each step it chooses the best, among the children of the current vertex except those in the tabu set, as the new current vertex (thereby recognizing cycles no larger than the size of the tabu set), updates the tabu set, and if n is better than n*, then it replaces n* with n.
- *Simulated annealing algorithm*: The choice of the next vertex is random. If the goal-function value of the chosen vertex (r) is better than that of the current vertex (n), then it moves there; if worse, then the probability of accepting the new vertex is inversely proportional to the difference f(n) - f(r). This proportion, moreover, changes continuously during the search: for the same difference it will choose the worse-valued vertex r with a higher probability at the beginning, and a lower one later.

6 Backtracking searches

It keeps track (in the global workspace) of the path leading from the start vertex to the current vertex (and the not-yet-tried edges branching off it); it can append a new (not-yet-tried) edge to the end of the tracked path or delete the last edge (rule of backtracking), and it applies backtracking only as a last resort. Backtracking makes it possible for a decision made earlier about moving forward to be changed. This is therefore a *modifiable control strategy*. Ordering and cutting heuristics can be built into the search. Both relate locally to the not-yet-tried edges leading out of the current vertex. Conditions for backtracking: dead end, dead-end junction, cycle, depth limit.

- BT1 (no cycle and depth limit checking) terminates on finite acyclic directed graphs, and if there is a solution, then it finds one.
- BT2 (general) terminates on δ-graphs, and if there is a solution within the depth limit, then it finds one.

It is easy to implement, has small memory requirement, always terminates, and if there is a solution (below the depth limit), it finds one. But it does not guarantee an optimal solution; it can correct a bad decision made early only after very many steps, and it can traverse a dead-end segment several times if that segment can be reached by several paths.

7 Graph searches

In its global workspace are the already explored paths starting from the start vertex (this is the so-called *search graph*), with those vertices of the paths separately marked whose successors we do not yet (or not well enough) know. These are the *open vertices*. The search rules expand an open vertex, that is, produce (or re-produce) all successors of the vertex. The control strategy strives to choose the most favorable open vertex, using an *evaluation function* (f) for this. Since an open vertex that is not chosen at a given moment can still be chosen later, a

modifiable control strategy is realized here. The search keeps track, for every vertex, of a path leading to it (with the help of π back-pointers), as well as the cost of the path (g). It forms these values during operation, when it first discovers the vertex or later finds a cheaper path to it. In both cases (when these values of the vertex have changed) the vertex becomes open. When an already-expanded vertex becomes open again, then for its already-discovered descendants the cost of the path designated by the back-pointers will not necessarily match the recorded value g, and it is also not certain that these values relate to the cheapest path found so far; that is, the correctness of the search graph may become corrupted.

- Uninformed graph searches: *depth-first graph search* ($f = -g$, $c(n, m) = 1$ for every edge (n, m)), *breadth-first graph search* ($f = g$, $c(n, m) = 1$), *uniform-cost graph search* ($f = g$)
- The f of heuristic graph searches builds on the heuristic function h, which estimates in each vertex the remaining optimal cost h*. Such are the *look-ahead graph search* ($f = h$), the *A algorithm* ($f = g+h, h \geq 0$), the *A* algorithm* ($f = g + h, h^* \geq h \geq 0$ – h admissible), the A^C *algorithm* ($f = g + h, h^* \geq h \geq 0$, $h(n) - h(m) \leq c(n, m)$ for every edge (n, m)), and the *B algorithm* (where, instead of $f = g + h$, $h \geq 0$, we use g to choose the vertex to be expanded from among those open vertices whose f value is smaller than the maximum of the f values of the vertices expanded so far).

On finite δ-graphs every graph search terminates, and if there is a solution, it finds one. Most of the notable graph searches also find a solution on infinitely large graphs, if there is a solution. (Exceptions are look-ahead search and depth-first graph search not using a depth limit.) The A*, A^C algorithms find an optimal solution, if there is a solution. The A^C algorithm expands a vertex at most once. We measure the memory requirement of a graph search by the number of expanded vertices, and its running time by the number of their expansions. (A vertex can generally be expanded several times, but in δ-graphs only finitely many times.) For the A* algorithm the running time depends, in the worst case, exponentially on the number of expanded vertices, but if we choose a heuristic for which we already obtain the A^C algorithm, then the running time will be linear. Of course, with this other heuristic the number of expanded vertices also changes, so it is not certain that an A^C algorithm performs, on the same graph, fewer expansions in total than an A* algorithm that expands a vertex several times. The running time of the B algorithm is quadratic, and if it uses a heuristic function like the A* algorithm (that is, an admissible one), then it likewise finds an optimal solution (if there is a solution) and the number of expanded vertices (and, incidentally, their set too) coincides with those expanded by the A* algorithm.

8 Two-player (perfect-information, zero-sum, finite) games

Games are usually described with a state space representation, and the state graph is represented as a tree. A *winning (or non-losing) strategy* is a principle by following which a player can give, to every move of the opponent, a response such that they win (do not lose) the game. One of the players certainly has a winning (non-losing) strategy. Searching for a winning (non-losing) strategy in the *game tree* can cause a combinatorial explosion, so instead of this, subtree evaluation is usually applied to determine the next good move. The *minimax* algorithm builds, from the current position, a part of the game tree, evaluates its leaves according to how favorable the positions they represent are to us or to the opponent, then, alternating level by level, propagates up to the parent vertex the minimum of the children's values on the opponent's levels and their maximum on our own levels. The vertex from which a value reaches the root will be our next move. The best-known modification of minimax is the *alpha-beta* algorithm, which on the one hand has a smaller memory requirement (it stores only one branch of the examined subtree at a time), and on the other hand, because of a specific cutting strategy, examines far fewer vertices than minimax. We specify an α value on our own level and a β value on the opponent's, initially with the values $-\infty$ and $+\infty$. When backtracking we change them, increasing α and decreasing β. A cut occurs if along the path there are values such that $\alpha \geq \beta$. Further modifications are the averaging one (we take the average of the largest m and smallest n values), and the minimax with alternating-depth evaluation (let the evaluation function show a realistic value on every branch, with a quiescence test), as well as the negamax algorithm (on the opponent's level the value times (-1), we choose the maximum on every level from the negated values).